

The rules-based-v2 Reference Policy in the ARC Disaster-Response Environment

ARC_Game RL — technical reference

June 24, 2026

Abstract

`rules-based-v2` (`improved_rules_based_decision` in `benchmark_models.py`) is a non-learning, API-free heuristic that repairs the four documented flaws of the baseline `rules-based` policy: it builds and hires *multiple* units per turn, invests in capacity *ahead* of demand (bounded by what it can staff), chooses build sites *geographically*, and *deploys* its budget rather than hoarding a fixed reserve. Its task choices use a value function with the motel’s recurring cost *priced in*, so relocations route into free shelters automatically. In an 8-episode smoke it scores ≈ 3.0 vs the baseline’s 2.5 (full food, 93% lodging, casework 0.43). Reward terms are defined in the companion *Reward Scoring* document.

Contents

1	Hardcoded Limits and Constants	1
2	Decision Pipeline	1
3	Stage 2: The Long-Term Choice Value Function	2
4	Stage 3: Staffing-Aware Capacity Target	3
5	Stage 4: Geographic, Multi-Build Construction	4
6	Stage 5: Multi-Hire	4
7	The Complete Policy	5

1 Hardcoded Limits and Constants

The policy has two kinds of constants: **policy hyperparameters** it chooses, and **environment limits** it must respect (these are enforced by the simulator / action enumerator, not by the policy). Table 1 lists both.

2 Decision Pipeline

Each round the policy returns task-choice selections C and a list of game-action indices X , built in five stages:

Symbol (code)	Value	Role
<i>Policy hyperparameters (set in <code>benchmark_models.py</code>)</i>		
B_{turn} (<code>_V2_BUILD_-PER_TURN</code>)	3	Max new buildings appended in one turn.
U_{max} (<code>_V2_PIPELINE_CAP</code>)	4	Max concurrent <i>unstaffed</i> buildings (UnderConstruction / NeedWorker) allowed before building pauses.
β (<code>_V2_OP_BUFFER</code>)	\$3000	Cash kept before any discretionary build/hire — sized to fund a reactive paid food airlift, <i>not</i> hoarded.
K_{tgt} (<code>_POT_KITCHEN_-TARGET</code>)	2	Target number of kitchens.
ρ (<code>_ROUNDS_PER_DAY</code>)	4	Rounds per in-game day (motel bills per day).
σ (<code>_V2_SHELTER_BEDS</code>)	10	Population capacity per shelter (for sizing).
δ_{shel} (in-code)	100	Penalty subtracted from a shelter choice that lacks space.
<i>Environment limits the policy respects (enforced by the simulator)</i>		
H_{day} (<code>DAILY_HIRE_LIMIT</code>)	5/day	Max workers hired per in-game day; hire actions bundle quantities 1–5.
ω (<code>BUILDING_-REQUIRED_WORKFORCE</code>)	4	Workers to operate a building (used to size staffing).
m (motel rate)	\$200/person/day	Recurring motel charge priced into the choice value.
w_c (<code>w_food_cost</code>)	0.0002	Reward cost weight, reused so the choice value matches the reward scale.

Table 1: Constants in `rules-based-v2`. Policy hyperparameters are tunable; environment limits are fixed by the simulator and merely respected by the policy.

1. **Reactive core** — inherit the greedy policy’s zero-cost worker assignments (staff buildings that need workers).
2. **Long-term task choices** — answer every open task with a choice scored by a value function that prices in the motel’s recurring cost (§3).
3. **Staffing-aware capacity target** — decide *what* to build, bounded by what can be staffed over the horizon (§4).
4. **Geographic, multi-build construction** — decide *where* and append several builds, paced by a staffable pipeline and the cash buffer (§5).
5. **Multi-hire** — hire the largest affordable batch to close the staffing gap (§6).

3 Stage 2: The Long-Term Choice Value Function

For an active task t with demand indicator $D(t) = \mathbf{1}[\text{taskType} \in \{\text{Demand}, \text{Emergency}\}]$, each choice c exposes a budget impact $b(c)$ and satisfaction impact $s(c)$. Choices are first *classified by destination* from the choice text $\tau(c)$ (lower-cased), guarding against a client group’s *name*

containing “Motel”/“Shelter”:

$$\text{cw}(c) = [\text{“casework”} \in \tau(c)], \quad (1)$$

$$\text{mo}(c) = [\text{“motel”} \in \tau(c)] \wedge \neg \text{cw}(c), \quad (2)$$

$$\text{sh}(c) = [\text{“shelter”} \in \tau(c)] \wedge \neg \text{mo}(c) \wedge \neg \text{cw}(c). \quad (3)$$

With $\text{cost}(c) = \max(0, -b(c))$ and the *acting* predicate $\alpha(c) = [\text{cost}(c) > 0] \vee [s(c) \geq 10]$, the value is

$$v(c) = \begin{cases} \frac{b(c)}{10^4} + 10^{-2}s(c), & b(c) > 0 \text{ (funding)} \\ \mathbf{1}[\alpha(c) \wedge D(t)] + 10^{-2}s(c) - w_c(\text{cost}(c) + R(c)), & b(c) \leq 0 \text{ (acting/waiting)} \end{cases} \quad (4)$$

where the key addition over the myopic greedy value is the **long-term recurring cost** of the motel, charged over the remaining days of the episode:

$$R(c) = \begin{cases} m \cdot q(c) \cdot \max(1, \text{roundsLeft}/\rho), & \text{mo}(c) \\ 0, & \text{otherwise,} \end{cases} \quad (5)$$

with $m = \$200/\text{person}/\text{day}$ and $q(c)$ the relocation quantity (`deliveryQuantity`, default 20). Because $w_c R(c)$ is large early (many days left) and shrinks to near zero late, shelters dominate the motel for most of the episode and the motel becomes acceptable only near the end — so *relocation routing into shelters emerges from the value function*, with no separate override. The *acting* predicate ensures free non-acting options (e.g. “request from kitchens”, which fails without a kitchen) score 0 and lose to the reliable paid action.

Per-task selection with a shelter-space guard. For each task the policy takes the highest-value choice, but first subtracts $\delta_{\text{shel}} = 100$ from any shelter choice that cannot fit the relocation (so it falls back to the motel rather than a deferred shelter delivery that would fail). Let g be the current free shelter space (sum over *InUse* shelters) and p_t the affected community’s population (default 20):

$$\hat{v}(c) = v(c) - \delta_{\text{shel}} \cdot \mathbf{1}[\text{sh}(c) \wedge g < p_t]. \quad (6)$$

The best choice $c^* = \arg \max_c \hat{v}(c)$ is committed iff $\hat{v}(c^*) > 0$ or it fulfils a real demand ($\alpha(c^*) \wedge D(t)$); if a shelter choice is taken, g is decremented by p_t (Algorithm 1).

4 Stage 3: Staffing-Aware Capacity Target

The policy invests *ahead* of demand, but never queues more capacity than it can *staff*: hiring is capped at $H_{\text{day}} = 5/\text{day}$ and each building needs $\omega = 4$ workers, so chasing the full displaced population would just create unstaffable buildings. Let W be the current total workers (free + working), and roundsLeft the rounds remaining; the staffable building budget is

$$d = \lceil \text{roundsLeft}/\rho \rceil \quad (\text{days left}), \quad W_{\text{max}} = W + H_{\text{day}} \cdot d, \quad (7)$$

$$N_{\text{max}} = \lfloor W_{\text{max}}/\omega \rfloor, \quad N_{\text{budget}} = \max(0, N_{\text{max}} - N_{\text{exist}}), \quad (8)$$

where $N_{\text{exist}} = n_{\text{cw}} + n_{\text{kit}} + n_{\text{shel}}$ counts all current buildings. The shelter shortfall toward the known displaced population P (sum of community populations) is $n_{\text{need}} = \lceil \max(0, P - \text{shelterCap})/\sigma \rceil$.

Algorithm 1: LONGTERMCHOICESELECTION

input : active tasks; free shelter space g ; roundsLeft
output : choices C

```
1  $C \leftarrow \emptyset$ 
2 foreach active task  $t$  with choices do
3    $D \leftarrow \mathbf{1}[\text{taskType} \in \{\text{Demand}, \text{Emergency}\}]; \quad p_t \leftarrow \text{affected community pop (default 20)}$ 
4    $c^* \leftarrow \arg \max_c \hat{v}(c)$  // Eq. (4) + shelter-space guard
5   if  $\hat{v}(c^*) > 0$  or  $c^*$  fulfils a demand then
6     add  $(t, c^*)$  to  $C$ 
7     if  $c^*$  is a shelter choice and  $g \geq p_t$  then
8        $g \leftarrow g - p_t$ 
9 return  $C$ 
```

The build wishlist is then assembled **kitchens before shelters** (food is a full reward point hard-gated by a kitchen) and truncated to the staffable budget:

$$\begin{aligned} \text{WANT} &= [\text{CASEWORKSITE}]_{n_{\text{cw}} < 1} \parallel [\text{KITCHEN}]^{(K_{\text{tgt}} - n_{\text{kit}})} \parallel [\text{SHELTER}]^{n_{\text{need}}}, \\ \text{WANT} &\leftarrow \text{WANT}[: N_{\text{budget}}] \quad (\text{truncate to the staffable budget}). \end{aligned} \tag{9}$$

5 Stage 4: Geographic, Multi-Build Construction

Where to build. Each available site s is ranked by distance from its world position to the nearest community, with a large penalty for flood-blocked routes (so shelters sit close to the communities they serve and avoid blocked sites):

$$\text{rank}(s) = \min_{c \in \text{communities}} \|\text{pos}(s) - \text{pos}(c)\|_2 + 10^6 \cdot \mathbf{1}[\text{name}(s) \in \text{blockedRoutes}]. \tag{10}$$

How many. The policy walks WANT in priority order and appends a build at the best-ranked unused site for each type, stopping when any guard trips: the per-turn cap $B_{\text{turn}} = 3$, the unstaffed-pipeline cap $U_{\text{max}} = 4$ (current unstaffed + queued), or the cash buffer (a build is only committed if $\text{cost} \leq \text{budget} - \beta$). Building is skipped entirely when fewer than 2 rounds remain.

6 Stage 5: Multi-Hire

The staffing gap counts workers needed by currently-understaffed buildings *plus* four per building queued this turn, minus the free pool f :

$$\text{gap} = \max\left(0, \sum_{b \in \text{NeedWorker}} (\text{required}(b) - \text{assigned}(b)) + \omega \cdot |\text{built}| - f\right). \tag{11}$$

If $\text{gap} > 0$, the policy appends the single `hire_untrained` action with the *largest* quantity whose cost fits the buffer ($\leq \text{budget} - \beta$). The action enumerator already caps the offered quantity at the remaining daily limit, so one action hires up to 5 workers — the most the environment permits in a day.

Algorithm 2: IMPROVEDRULESBASEDPOLICY (*improved_rules_based_decision*)

```
input : game state; round  $r$ ; total rounds  $T$ ; roundsLeft =  $\max(0, T - r)$ 
output: choices  $C$ ; action indices  $X$ 
1  $X \leftarrow$  greedy worker assignments (reactive core)
2  $C \leftarrow$  LONGTERMCHOICESELECTION( $\cdot$ ) // Algorithm 1
3 compute  $P$ , shelterCap,  $n_{cw}$ ,  $n_{kit}$ ,  $n_{shel}$ ,  $W$ , budget
4  $N_{\text{budget}} \leftarrow \max(0, \lfloor (W + 5 \lceil \text{roundsLeft} / \rho \rceil) / \omega \rfloor - N_{\text{exist}})$ 
5  $\text{WANT} \leftarrow ([\text{CASEWORK}]_{n_{cw} < 1} \| [\text{KITCHEN}]^{K_{\text{tgt}} - n_{\text{kit}}} \| [\text{SHELTER}]^{n_{\text{need}}})[: N_{\text{budget}}]$ 
6 rank available sites by rank( $s$ )
7 if roundsLeft  $\geq 2$  then
8   foreach  $btype \in \text{WANT}$  do
9     if  $|built| \geq B_{\text{turn}}$  or  $unstaffed + |built| \geq U_{\text{max}}$  then
10      break
11      $s \leftarrow$  best-ranked unused site offering  $btype$ 
12     if  $s$  exists and  $\text{cost}(s) \leq \text{budget} - \beta$  then
13       append build( $s$ ) to  $X$ ;  $\text{budget} -= \text{cost}(s)$ ; mark  $s$  built
14 gap  $\leftarrow \max(0, \text{need}_{\text{now}} + \omega|built| - f)$ 
15 if gap  $> 0$  then
16   append the largest-quantity hire_untrained with cost  $\leq \text{budget} - \beta$  to  $X$ 
17 return ( $C, X$ )
```

7 The Complete Policy

Empirical note. On an 8-episode smoke (18 rounds), *rules-based-v2* scored ≈ 3.01 vs the baseline *rules-based* 2.49 and the best LLM (gpt-5.4) 2.25: full food (1.0), lodging 0.93 (93% fulfilment vs the baseline’s 53%), casework 0.43 (vs 0.32), at equal cost-efficiency and a healthier final budget. Numbers are a smoke, not a final benchmark.